

common_reference_t of reference_wrapper Should Be a Reference Type

Document #: P2655R3
Date: 2023-02-07
Project: Programming Language C++
Audience: SG9, LEWG
Reply-to: Hui Xie
<hui.xie1990@gmail.com>
S. Levent Yilmaz
<levent.yilmaz@gmail.com>
Tim Song
<t.canens.cpp@gmail.com>

Contents

- 1 Revision History
 - 1.1 R3
 - 1.2 R2
 - 1.3 R1
 - 1.4 R0
- 2 Abstract
- 3 Motivation and Examples
- 4 Design
 - 4.1 Why Should the Result be `T&` and not `reference_wrapper<T>`
 - 4.2 Alternatives Considered
 - 4.2.1 Option 1: Support Exact Same Type with CV-Ref Variations
 - 4.2.2 Option 2: Treat `reference_wrapper<T>` as `T&`
 - 4.3 Supporting All Compatible Conversions (Option 3)
 - 4.4 Supporting `cv`-qualified `reference_wrapper` and Other Proxy Types
 - 4.4.1 The Issue with `cv`-qualified Proxy Types
 - 4.4.2 The Fix
- 5 Implementation Experience
- 6 Wording
 - 6.1 Feature Test Macro
- 7 References

§ 1 Revision History

§ 1.1 R3

- Added new feature test macro and renamed the old one

§ 1.2 R2

- Address issues with `common_reference` of any cv-qualified proxy types.
- Replaced Wording paragraphs with code.

§ 1.3 R1

- Added reasons why the result should be $\tau\&$
- Support `const` and `volatile`
- Support derive-base conversions

§ 1.4 R0

- Initial revision.

§ 2 Abstract

This paper proposes a fix that makes the `common_reference_t<T&, reference_wrapper<T>>` a reference type $\tau\&$.

§ 3 Motivation and Examples

C++20 introduced the meta-programming utility `common_reference` 21.3.8.7 [meta.trans.other] in order to programmatically determine a common reference type to which one or more types can be converted or bound.

The precise rules are rather convoluted, but roughly speaking, for given two non-reference non-cv qualified types X and Y , `common_reference<X&, Y&>` is equivalent to the expression `decltype(false ? x : y)` where x and y are qualified $X\&$ and $Y\&$, respectively, provided the ternary expression is valid. (cv-qualified references are treated differently, and is explained below in [Section 4.4](#).) Otherwise, `basic_common_reference` trait is consulted, which is a customization point that allows users to influence the result of `common_reference` for user-defined types. (Two such specializations are provided by the standard library, namely, for `std::pair` and `std::tuple` which map `common_reference` to their respective elements.) And if no such specialization exists, then the result is `common_type<X, Y>`.

The canonical use of `reference_wrapper<T>` is its being a surrogate for $\tau\&$. One might expect that the ternary operator would yield a $\tau\&$, but due to language rules, that is not quite the case:

```
int i = 1, j = 2;
std::reference_wrapper<int> jr = j; // ok - implicit constructor
int & ir = std::ref(i); // ok - implicit conversion
int & r = false ? i : std::ref(j); // error - conditional expression is ambiguous.
```

The reason for the error is not because `i` and `ref(j)`, an `int&` and a `reference_wrapper<int>`, are incompatible. It is because they are too compatible! Both types can be converted to one another, so the type of the ternary expression is ambiguous.

Hence, per the current rules of `common_reference` as summarized above, and with the lack of any `basic_common_reference` specialization, the evaluation falls back to `common_type<T,`

reference_wrapper<T>>, whose ::type is valid and equal to T. In other words, common_reference determines that the reference type to which both T& and a reference_wrapper<T> can bind is a prvalue T!

The authors believe this current determination logic for common_reference for an lvalue reference to a type T and its reference_wrapper<T> is merely an accident, and is incompatible with the canonical purpose of the reference_wrapper. The answer should have been T&. (Note that, there is no ambiguity with a reference_wrapper<T> and rvalue of T, since former is convertible to latter, but not vice versa.)

This article proposes an update to the standard which would change the behavior of common_reference to evaluate as T& given T& and an a reference_wrapper<T>, commutatively. Any evolution to implicit conversion semantics of reference_wrapper, or of the ternary operator for that matter, is out of the question. Therefore, the authors propose to implement this change via providing a partial specialization of basic_common_reference trait.

Below are some motivating examples:

C++20	Proposed
<pre>static_assert(same_as< common_reference_t<int&, reference_wrapper<int>>, int>); static_assert(same_as< common_reference_t<int&, reference_wrapper<int>&>, int>); static_assert(same_as< common_reference_t<int&, const reference_wrapper<int>&>, const int>);</pre>	<pre>static_assert(same_as< common_reference_t<int&, reference_wrapper<int>>, int&>); static_assert(same_as< common_reference_t<int&, reference_wrapper<int>&>, int&>); static_assert(same_as< common_reference_t<int&, const reference_wrapper<int>&>, int&>);</pre>
<pre>class MyClass { vector<vector<Foo>> foos_; Foo delimiter_; public: void f() { auto r = views::join_with(foos_, views::single(std::ref(delimiter_))); for (auto&& foo : r) { // foo is a temporary copy } } };</pre>	<pre>class MyClass { vector<vector<Foo>> foos_; Foo delimiter_; public: void f() { auto r = views::join_with(foos_, views::single(std::ref(delimiter_))); for (auto&& foo : r) { // foo is a reference to the // original element } } };</pre>
<pre>class MyClass { vector<Foo> foo1s_; Foo foo2_; public: void f() { auto r = views::concat(foo1s_, views::single(std::ref(foo2_))); for (auto&& foo : r) { // foo is a temporary copy } } };</pre>	<pre>class MyClass { vector<Foo> foo1s_; Foo foo2_; public: void f() { auto r = views::concat(foo1s_, views::single(std::ref(foo2_))); for (auto&& foo : r) { } } };</pre>

C++20	Proposed
<pre> } }; </pre>	<pre> // foo is a reference to the // original element } }; </pre>

In the second and the third example, the user would like to use `views::join_with` and `views::concat` [P2542R2], respectively, with a range of `Foo`s and a single `Foo` for which they use a `reference_wrapper` to avoid copies. Both of the range adaptors rely on `common_reference_t` in their respective implementations (and specifications). As a consequence, the counter-intuitive behavior manifests as shown, where the resultant views' reference type is a prvalue `Foo`. There does not seem to be any way for the range adaptor implementations to account for such use cases in isolation.

§ 4 Design

§ 4.1 Why Should the Result be $\tau\&$ and not `reference_wrapper<T>`

As they can both be converted to each other, the result of `common_reference_t` can be either of them in theory. However, the authors believe that the users would expect the result to be $\tau\&$. Given the following example,

```

auto r = views::concat(foos,
    views::single(std::ref(foo2_)));
for (auto&& foo : r) {
    foo = anotherFoo;
}

```

If the result is `reference_wrapper<T>`, the assignment inside the for loop would simply rebind the `reference_wrapper` to a different instance. On the other hand, if the result is $\tau\&$, the assignment would call the copy assignment operator of the original `foos`. The authors believe that the latter design is the intent of code and is the natural choice.

§ 4.2 Alternatives Considered

The following are some of the alternatives that considered originally. But later dropped in favor of the one discussed in the next section.

§ 4.2.1 Option 1: Support Exact Same Type with CV-Ref Variations

One option would be to provide customisations for only `reference_wrapper<T>` and cv-ref τ . Note that this version is rather restrictive:

```

template <class T, class U, template <class> class TQual,
    template <class> class UQual>
    requires std::same_as<T, remove_cv_t<U>>
struct basic_common_reference<T, reference_wrapper<U>, TQual, UQual> {
    using type = common_reference_t<TQual<T>, U&>;
};

template <class T, class U, template <class> class TQual,
    template <class> class UQual>
    requires std::same_as<remove_cv_t<T>, U>
struct basic_common_reference<reference_wrapper<T>, U, TQual, UQual> {
    using type = common_reference_t<T&, UQual<U>>;
};

```

§ 4.2.2 Option 2: Treat `reference_wrapper<T>` as `T&`

This options completely treats `reference_wrapper<T>` as `T&` and delegates `common_reference<reference_wrapper<T>, U>` to the `common_reference<T&, U>`. Therefore, it would support any conversions (including derived-base conversion) that `T&` can do.

```
template <class T, class U, template <class> class TQual, template <class> class UQual>
    requires requires { typename common_reference<TQual<T>, U&::type; }
struct basic_common_reference<T, reference_wrapper<U>, TQual, UQual> {
    using type = common_reference_t<TQual<T>, U&};
};

template <class T, class U, template <class> class TQual, template <class> class UQual>
    requires requires { typename common_reference<T&, UQual<U>::type; }
struct basic_common_reference<reference_wrapper<T>, U, TQual, UQual> {
    using type = common_reference_t<T&, UQual<U>};
};
```

Immediately, it run into ambiguous specialisation problems for the following example

```
common_reference_t<reference_wrapper<int>, reference_wrapper<int>>;
```

A quick fix is to add another specialisation

```
template <class T, class U, template <class> class TQual, template <class> class UQual>
    requires requires { typename common_reference<T&, U&::type; }
struct basic_common_reference<reference_wrapper<T>, reference_wrapper<U>, TQual, UQual>
{
    using type = common_reference_t<T&, U&};
};
```

However, this has some recursion problems.

```
common_reference_t<reference_wrapper<reference_wrapper<int>>,
    reference_wrapper<int>&>;
```

The user would expect the above expression to yield `reference_wrapper<int>&>`. However it yields `int&` due to the recursion logic in the specialisation.

And even worse,

```
common_reference_t<reference_wrapper<reference_wrapper<int>>,
    int&>;
```

The above expression would also yield `int&` due to the recursion logic, even though the nested `reference_wrapper` is not convertible to `int&`.

The rational behind this option is that `reference_wrapper<T>` behaves exactly the same as `T&`. But does it?

There is conversion from `reference_wrapper<T>` to `T&`, and if the result requires another conversion, the language does not allow `reference_wrapper<T>` to be converted to the result.

This would cover majority of the use cases. However, this does not cover the derive-base conversions, i.e. `common_reference_t<reference_wrapper<Derived>, Base&>>`. This is a valid use case and the authors believe that it is important to support it.

§ 4.3 Supporting All Compatible Conversions (Option 3)

The above exposure can be extrapolated to any cv-qualified or other cross-type compatible conversions. That is, if `common_reference_t<U, V>` exists then `common_reference_t<reference_wrapper<U>, V>` and

`common_reference_t<U, reference_wrapper<V>>` should also exist and be equal to it, *given* the only additional requirement that `reference_wrapper<U>` or `reference_wrapper<V>`, respectively, can be also implicitly converted to `common_reference_t<U, V>`. This statement only applies when the evaluation of `common_reference_t` falls through to `basic_common_reference` (see next section).

The authors propose to support such behavior by allowing `basic_common_reference` specialization to delegate the result to that of the `common_reference_t` of the wrapped type with the other non-wrapper argument. Furthermore, impose additional constraints on this specialization to make sure that the `reference_wrapper` is convertible to this result.

In order to support commutativity, we need to introduce two separate specializations, and further constrain them to be mutually exclusive in order avoid ambiguity.

Finally, we have to explicitly disable the edge cases with nested `reference_wrappers` since, while `reference_wrapper<reference_wrapper<T>>` is not convertible to `T&`

§ 4.4 Supporting cv-qualified `reference_wrapper` and Other Proxy Types

§ 4.4.1 The Issue with cv-qualified Proxy Types

As implied in the previous sections, the rules of the `common_reference` trait are such that any `basic_common_reference` specialization is consulted only if some ternary expression of the pair of arguments is ill-formed (see 21.3.8.7 [meta.trans.other]/5.3.1).

More precisely, that ternary expression is denoted by `COMMON-REF(T1, T2)`, where `T1` and `T2` are the two arguments of the trait, and `COMMON-REF` is a complex macro defined in 21.3.8.7 [meta.trans.other]/2. For the cases where both `T1` and `T2` are lvalue references, their `COMMON-REF` is the union of their cv-qualifiers applied to both. For example, given `T1` is `const X&` and `T2` is `Y&` (where `X` and `Y` are non-reference types), the evaluated expression is `decltype(false ? xc : yc)` where `xc` and `yc` are `const X&` and `const Y&`, respectively. Note that, the union of cv-qualifiers is `const` and it is applied to *both arguments* even though originally `T2` is a non-const reference.

The origin and rationale for these contrived rules are rather obscure. But one consequence in the context of this paper is that there are interesting edge cases where the `basic_common_reference` treatment do not apply. Take,

```
int i = 3;
const std::reference_wrapper<int> r = i;
int& j = r; // ok.
```

That is, any cv qualification of `reference_wrapper<int>` itself does not change its semantics, since it is just a proxy to an `int&`. So, it would be natural to expect that `int&` should be the common reference of `int&` and `const reference_wrapper<int>&`, since objects of both types can be assigned to an `int&`.

However, because of the way `COMMON-REF` is defined, the evaluated ternary expression is `decltype(false ? r : jc)`, where `jc` is `const int&`. Lo and behold, this expression is no longer ill-formed and evaluates to `const int&` (the conversion direction is no longer ambiguous, since `reference_wrapper<int>` can not be constructed from an `int const&`), and we get:

```
// in the current standard, with or without the basic_common_reference
// specialization of this proposal:
static_assert(std::same_as<
    std::common_reference_t<
        const std::reference_wrapper<int>&,
        int&
```

```

    >,
    const int& // not int& !!
>);

```

This issue exists not only in `reference_wrapper`, but any proxy-like types with reference cast operators. For example,

```

struct A {};

struct B {
    operator A& () const;
};

```

Even though the builtin ternary operator `?:` does return the expected type `A&`,

```

A a;
const B b;
static_assert(std::same_as<
    decltype(false? a : b),
    A&
>);

```

`common_reference_t` surprisingly results in `const A&`

```

static_assert(std::same_as<
    std::common_reference_t<A&, const B&>,
    const A& // not A& !!
>);

```

§ 4.4.2 The Fix

Per SG9's direction, we'd like to fix this issue along with the `basic_common_reference` treatment in this paper. Let's revisit the precise rules of `common_reference` trait [\[meta.trans.other#5.3\]](#). Its member `::type` is,

1. *COMMON-REF* if not ill-formed.
2. Otherwise, `basic_common_reference` if a specialization exists.
3. Otherwise, `decltype` of ternary operator `?:`.
4. Otherwise, `common_type`.
5. Otherwise, does not exist.

The reason why `common_reference_t<const reference_wrapper<int>&, int&>` does not use the `basic_common_reference` specialization and why `common_reference_t<A&, const B&>` does not use the ternary operator `?:` is that Step-1 *COMMON-REF* is well formed. But it yields an undesired result.

Step-1 is important to have before the customization Step-2, because the *COMMON-REF* layer provides a generalized mechanism to handle reference cases before the user-specializable component. This makes the `common_reference` trait more convenient for users, and more importantly, harder to get wrong. For example, `common_reference<tuple<int>&, tuple<int>&>` should remain `tuple<int>&` and not `tuple<int&>` as would a straightforward `basic_common_reference` specialization for tuples yield (which is the current one in the standard [\[tuple#common.ref\]](#)). It would be unreasonable to expect each specialization to handle every reference combination correctly, exhaustively, and consistently.

Yet, *COMMON-REF* was probably not meant to deal with proxy references and user-defined conversions at all. It is used only when the types involved are reference types, and does not make any sense if it were meant to handle things that are convertible to reference types.

To rectify this situation, the authors propose to reject user-defined conversions entirely from Step-1. This would then allow Step-2 to provide the required custom semantics for any underlying proxy reference types where desired, and Step-3 to recover the ternary-operator based fallback.

This suggestion can be realized by an additional constraint on Step-1: Require a valid conversion to exist between each respective pointer types of the pair of arguments to the evaluated *COMMON-REF* result. Precise implementation can be found in the Wording section.

§ 5 Implementation Experience

- The authors implemented the proposed wording below without any issue[[ours](#)].
- The authors also applied the proposed wording in LLVM’s libc++ and all libc++ tests passed.[[libcxx](#)]

§ 6 Wording

Modify 21.3.8.7 [[meta.trans.other](#)] section (5.3.1) as

- (5.3.1) Let *R* be *COMMON-REF*(*T1*, *T2*). If *T1* and *T2* are reference types, ~~and *COMMON-REF*(*T1*, *T2*)~~ *R* is well-formed, and is convertible v<add_pointer t<T1>, add_pointer t<R>> && is convertible v<add_pointer t<T2>, add_pointer t<R>> is true, then the member typedef type denotes ~~that type~~ *R*.

Modify 22.10.2 [[functional.syn](#)] to add to the end of `reference_wrapper` section:

```
// [refwrap.common.ref] common_reference related
specializations
template <class R, class T, template <class> class RQual, template <class> class TQual>
requires see below
struct basic_common_reference<R, T, RQual, TQual>;

template <class T, class R, template <class> class TQual, template <class> class RQual>
requires see below
struct basic_common_reference<T, R, TQual, RQual>;
```

Add the following subclause to 22.10.6 [[refwrap](#)]:

§ ??? common_reference related specializations [[refwrap.common.ref](#)]

```
template <class T>
inline constexpr bool is-ref-wrapper = false; // exposition only

template <class T>
inline constexpr bool is-ref-wrapper<reference_wrapper<T>> = true;

template<class R, class T, class RQ, class TQ>
concept ref-wrap-common-reference-exists-with = // exposition only
    is-ref-wrapper<R>
    && requires {
        typename common_reference_t<typename R::type&, TQ>;
    }
    && convertible_to<RQ, common_reference_t<typename R::type&, TQ>>
    ;

template <class R, class T, template <class> class RQual, template <class> class TQual>
requires( ref-wrap-common-reference-exists-with<R, T, RQual<R>, TQual<T>>
    && !ref-wrap-common-reference-exists-with<T, R, TQual<T>, RQual<R>> )
struct basic_common_reference<R, T, RQual, TQual> {
    using type = common_reference_t<typename R::type&, TQual<T>>;
```



```
};

template <class T, class R, template <class> class TQual, template <class> class RQual>
    requires( ref-wrap-common-reference-exists-with<R, T, RQual<R>, TQual<T>>
               && !ref-wrap-common-reference-exists-with<T, R, TQual<T>, RQual<R>> )
struct basic_common_reference<T, R, TQual, RQual> {
    using type = common_reference_t<typename R::type&, TQual<T>>;
};
```

§ 6.1 Feature Test Macro

Add the following macro definition to 17.3.2 [version.syn], header <version> synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_common_reference 20XXXXL // also in <type_traits>
#define __cpp_lib_common_reference_wrapper 20XXXXL // also in <functional>
```

§ 7 References

[libcxx] Hui Xie and S. Levent Yilmaz. Implementation of common_reference in libc++.

<https://reviews.llvm.org/D141200>

[ours] Hui Xie and S. Levent Yilmaz. A proof-of-concept implementation of common_reference_t for reference_wrapper.

https://github.com/huixie90/cpp_papers/tree/main/impl/ref_wrapper

[P2542R2] Hui Xie, S. Levent Yilmaz. 2022-05-11. views::concat.

<https://wg21.link/p2542r2>